HTTP API

The server speaks one wire format for songs: `.sng`, whatever shape a song was found in on disk. That is the design commitment in ADR-001 — an unmodified YARG reads a `.sng` natively, so a client can write ordinary files into an ordinary songs folder and the game needs no change at all.

Everything below is v1 and unversioned only in the sense that `/api/v1` is the version.

This API has a first-party consumer. `yarg-sync` (`cmd/yarg-sync`, documented in [`SYNC-CLIENT.md`]) is built against it, and `internal/e2e` runs the real client against the real handler on every pipeline. Three things below are load-bearing for it rather than incidental, so changing them breaks a shipped binary:

- **`POST /api/v1/have`** is how the client takes inventory in one round trip, and its empty-list form is how `-prune` learns the server's full set.
- **300 Multiple Choices** on a shared chart hash. The client relies on the server *not* choosing, and resolves it deterministically itself so that two machines syncing the same library pick the same package. (Picking the same package is necessary for a shared library; it is not sufficient. Until 2026-09-06 the two machines picked the same package and still received different bytes, because packing itself was not deterministic.)
- **`ETag` as the package hash, and `Range`.** Identity is re-derived from the received bytes client-side, so a substituted or truncated response fails closed. Both of these rest on packing being deterministic — one `ETag` must mean one sequence of octets, or a resume splices two different archives together. It did not hold until 2026-09-06; see `docs/TEST-CORPUS.md`.

Endpoints

Method	Path	What it is for
`GET`	`/`	The browse page, when `browse_ui` is on.
`GET`	`/healthz`	Liveness. Returns `ok`.
`GET`	`/version`	The build's version string.
`GET`	`/api/v1/library`	What was indexed, and what could not be.

Method	Path	What it is for
`GET`	`/api/v1/songs`	Browse and search.
`GET`	`/api/v1/songs/{chart_hash}`	Every package sharing a chart hash.
`POST`	`/api/v1/have`	Bulk "what am I missing".
`GET`	`/song/{chart_hash}.sng`	The bytes.

`GET /api/v1/library`

```
{
  "songs": 22,
  "distinct_charts": 22,
  "duplicate_packages": 0,
  "built_at": "2026-09-05T20:11:05Z",
  "problems": [],
  "sort_attributes": ["name", "artist", "album", "artist_album", "genre", "subgenre",
                      "year", "charter", "playlist", "source", "song_length", "date_added"]
}
```

`songs` counts packages; `distinct_charts` counts songs as YARG identifies them, and the two differ whenever two packages share a chart.

`problems` is the important field. It names every directory or archive the scan could not read. A library that quietly indexes 9,000 of 10,000 songs is indistinguishable from one that has 9,000 songs, and an operator has no way to find out which — so failures are surfaced here rather than logged once at start and forgotten.

`GET /api/v1/songs`

Parameter	Default	Meaning
`q`	—	Free text. Matched against name, artist, album, genre, subgenre, charter, source and playlist.
`sort`	`name`	One of the twelve attributes above. Anything else is a **400**.
`order`	`asc`	`asc` or `desc`. Anything else is a 400.
`limit`	50	Capped at 500.
`offset`	0	Past the end is an empty page, not an error.

```
{ "total": 22, "offset": 0, "limit": 50, "sort": "artist", "order": "asc",
  "query": "", "songs": [ /* catalog entries */ ] }
```

An unrecognised `sort` is refused rather than silently replaced with the default. A client that asked for one order and was handed another without being told has no way to notice.

On ordering. Results come back in the order the *client* would show them: values are normalised exactly as YARG.Core's `SortString` does — leading article dropped, diacritics folded, rich-text markup stripped — and the tie-breakers are upstream's own comparer chains. See `internal/sortkey` and ADR-002 for what is reproduced and for the two places this server deliberately differs.

On matching. Only the *folding* is the client's, so `q=bjork` finds "Björk" and `q=the beatles` finds "The Beatles". The matching itself — a substring test across those eight attributes — is this server's own. Upstream's search logic has not been read and no parity is claimed for it. A query does not match across a field boundary: `q=blur blur` finds nothing even when the artist and the album are both "Blur".

`GET /api/v1/songs/{chart_hash}`

Returns a **list**, because song identity is deliberately many-to-one:

```
{ "chart_hash": "0856db78…", "count": 2, "songs": [ /* … */ ] }
```

Two packages with the same chart and different audio are the same song to YARG — its own cache is `hash -> List<SongEntry>`. Returning only the first would hide the other from every client that asked.

`POST /api/v1/have`

The client sends the chart hashes it already holds; the answer is everything in the library that was not in that list. One round trip, whatever the size of either side.

```
{ "chart_hashes": ["0856db78…", "b434fc60…"] }
```

```
{ "library_total": 22, "missing": ["2db04926…", "…"], "missing_count": 20 }
```

Hashes are compared case-insensitively and surrounding whitespace is ignored, because a client assembling a list from its own cache should not be punished for either. `missing` is sorted, so the same question twice gives the same bytes.

Chart hash, not package hash, on purpose: a client that has the chart can play the song, and re-sending it a package that differs only in album art is bandwidth spent on nothing.

An unknown field in the body is a **400** rather than being ignored — a client that sent `chart_hash` for `chart_hashes` would otherwise be told, plausibly and wrongly, that it is missing the entire library.

`GET /song/{chart_hash}.sng`

The package bytes, always as `.sng`. A song stored as anything else — a loose folder, a `.zip` or a `.7z` — is packed on demand and the archive is cached; packing copies the chart byte for byte, so the song's identity is unchanged by it. The three shapes pack to the same bytes, so re-zipping a library does not change what a client downloads.

- `ETag` is the package hash — a hash of the content, so it is the same on every server holding the same package and survives a rescan, a restart and a cache wipe.
- `Range` is supported, so an interrupted download resumes rather than starting again.
- **300 Multiple Choices** when the chart hash is shared by several packages. The response lists them; `?package=<package_hash>` names one. The server does not choose, because choosing would hand different clients different audio for the same request.
- **404** when the hash is unknown, and also when the index points at a file that has since moved — with a message saying to rescan, because that is the fix.

What is not here yet

Ingest (`POST` of a folder, `.sng` or `.zip`) and authentication. See `docs/ROADMAP.md`. The server is currently read-only and unauthenticated: run it on a LAN, not on the internet.

Settings are a flag or a `key = value` line in `./yarg-song-server.conf`, same name for both, flag wins; `--write-config` prints a commented example. An unknown setting in that file is an error rather than a warning, on the same reasoning as the unknown-field rule on `POST /api/v1/have`.

`GET /`

A phone-friendly page listing the library: search, the same twelve sort attributes the API offers, and a per-song panel with metadata, parts, any issues the scanner flagged, and a download link. It is served from the binary — no CDN, no build step, nothing external — because the deployment this is aimed at is a Pi on a LAN at a party, where a page that needs the internet fails exactly when it is wanted.

It reads the sort attributes from `/api/v1/library` rather than hardcoding them, so the page cannot drift from the server that served it.

On by default; `browse_ui = no`, or `--browse-ui=false`, turns it off. It exposes nothing `/api/v1/songs` does not already serve unauthenticated to anyone who can reach the port — the decision that matters is the one in ADR-001, that this server is read-only and must not be exposed publicly.

It is registered as `GET /{$}`, an exact match on the root. A bare `GET /` in Go's ServeMux is a catch-all: it would answer every unmatched path with this page and a 200, turning every 404 documented above into an HTML page that a sync client would try to parse as a `.sng`. `TestBrowsePageOnAndOff` pins that, and fails if the pattern is ever loosened.

What it deliberately does not have is a queue. Upstream's open request #860 asks for search *and queueing* from a phone while YARG is running; queueing needs something inside the running game to read a queue, which is Phase 3 work. A button that cannot work is worse than no button, so there is not one.
